

Revision 03-04 · Levels of Inference

Engineering + Math for Memory, Throughput & Speedups

Interactive revision of [lesson 03](#) and [lesson 04](#): each question as asked, the answer, where I slipped, and the one-line takeaway.

Q1 · API bill with and without prompt caching PARTIAL

Covers: 03

QUESTION

Each request sends the same 3,000-token system instructions, a 200-token user question, and gets a 300-token answer. DeepSeek-R1 prices per million tokens: input \$0.55, cached input \$0.14, output \$2.19. What do 1,000 requests cost without caching, and with the system instructions cached?

Answer

	No cache	System prompt cached
System, 3,000 tokens	$3,000 \times 0.55 = 1,650$	$3,000 \times 0.14 = 420$
Question, 200 tokens	$200 \times 0.55 = 110$	$200 \times 0.55 = 110$
Answer, 300 output tokens	$300 \times 2.19 = 657$	$300 \times 2.19 = 657$
Sum (tokens $\times$ \$/M)	2,417	1,187
1,000 requests = sum $\div 10^6 \times 10^3$	\$2.42	\$1.19

Caching the shared prefix halves the bill; output (657) is now over half of what's left, so capping output length is the next lever.

RE-DRILL

Before multiplying, label each token count with its type: cached input, fresh input, or output.

Q2 · Personalization that kills the cache SOLID

Covers: 03, 11, 14

QUESTION

A teammate puts the user's name and the current timestamp at the very start of the prompt, before the 3,000-token system instructions. Overnight, the cached-input share drops to almost zero. Why, and what's the fix?

Answer

- The first tokens now differ for every request, so no request shares a prefix with an earlier one: nothing can be served at the cached-input price, and the bill jumps back toward \$2.42 per 1k.
- **Why prefixes:** with causal attention, each token's keys and values depend on every token before it. Change token 0 and every later token's K/V changes, so nothing after the first difference is reusable.
- **Fix:** stable content first (system instructions), variable content last (name, timestamp, user question).

Q3 · Does Llama 3 70B fit on one H100? **SOLID**

Covers: 04, 18, 19

QUESTION

Serve Llama 3 70B on one 80 GB H100. How big are the weights in BF16, FP8 and INT4, and does each one "fit" for serving users?

Answer (budget 72 GB = 90% of 80; ~3 GB runtime + activations; 320 KiB KV per token)

Precision	Weights = 70B × bytes	Verdict
BF16 (2 B)	140 GB	Doesn't load (> 80 GB)
FP8 (1 B)	70 GB	Loads, but $72 - 70 - 3 < 0$: no KV cache , can't serve
INT4 (0.5 B + scales)	~35 GB	$72 - 35 - 3 \approx 34$ GB KV ÷ 320 KiB ≈ ~100k tokens (~25 users at 4k)

Q4 · A 3× faster attention kernel (Amdahl) **PARTIAL**

Covers: 04

QUESTION

Attention takes 30% of a decode step. A teammate makes the attention kernel 3× faster and claims "decode is now 3× faster." What's the real speedup, and the best possible if attention became free?

How I solved it (time accounting)

1. Today a step = **100 units**: 30 attention + 70 everything else.
2. **3× faster attention**: $30 \div 3 = 10$ units; the other 70 don't change. New step = $10 + 70 = 80$.
Speedup = $100 \div 80 = 1.25\times$.
3. **Attention free**: $0 + 70 = 70$ units. Speedup = $100 \div 70 \approx 1.43\times$, the ceiling.
4. Formula: $S = 1 \div ((1 - p) + p/s) = 1 \div (0.7 + 0.3/3) = 1 \div 0.8 = 1.25$; with $s \rightarrow \infty$, $S \rightarrow 1 \div 0.7 \approx 1.43$.

RE-DRILL

Always redo it as "100 units → how many units now?" before reaching for the formula.

Exercise · Amdahl calculator for several optimizations SOLID

Covers: 04

QUESTION

Write `overall_speedup(parts)` where each part is (fraction of the original step, speedup of that part), non-overlapping. Return the true end-to-end speedup, which catches the "adding speedups" trap.

The structure: old time = 1.0 → each touched part now takes $\text{fraction} \div \text{speedup}$ → the untouched fraction keeps its full time → new time = the sum → $\text{speedup} = \text{old} \div \text{new}$.

```
def overall_speedup(parts):
    old_time = 1.0 # Step 1
    touched_new = sum(fraction / speedup for fraction, speedup in parts.values()) # Step 2
    untouched = 1 - sum(fraction for fraction, _ in parts.values()) # Step 3
    new_time = touched_new + untouched # Step 4
    return old_time / new_time # Step 5

overall_speedup({"attention": (0.3, 3)}) # 1.25
overall_speedup({"attention": (0.3, 3), "mlp": (0.5, 2)}) # 1.82 (not 5, not 6)
overall_speedup({"attention": (0.3, 1e9)}) # 1.43 (the ceiling)
```

Why 1.82: new time = $0.3/3 + 0.5/2 + 0.2 = 0.1 + 0.25 + 0.2 = 0.55$, and $1 \div 0.55 \approx 1.82$.

Optimizing 80% of the step still leaves 0.55 of the original time.